

Assignment 4

Harivdan Shrihari Narayanshastri - 125M1K010 – ADVT Lab

Title: Visualize the clusters for any synthetic dataset, implement the program for converting the clusters into histograms

Aim

To generate a synthetic 2D dataset using `make_blobs`, apply K-Means clustering to identify 3 clusters, visualize the clusters with scatter plot showing data points and centroids, export clustered data to CSV, and analyse cluster-wise histograms.

Dataset

- Type: Synthetic 2D dataset generated programmatically.
- Kaggle reference: 2D Clustering Dataset Collection (<https://www.kaggle.com/datasets/samoilovmikhail/2d-clustering-dataset-collection>)
- Parameters:
 - `n_samples=300` data points
 - `centers=3` clusters
 - `cluster_std=1.2` (standard deviation within clusters)
- Exported file: `clustered_data.csv` with columns Feature1, Feature2, Cluster (0, 1, 2).

Software and Libraries

- Python 3.x: numpy, pandas, matplotlib, scikit-learn (`make_blobs`, `KMeans`)

Code

```
# Assignment 4: K-Means Clustering on Synthetic Dataset
```

```
# Reference: Kaggle 2D Clustering Dataset Collection
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
from sklearn.datasets import make_blobs
```

```
from sklearn.cluster import KMeans
```

```
import pandas as pd
```

```
# Step 1: Generate synthetic 2D dataset
```

```
X, y_true = make_blobs(  
    n_samples=300,  
    centers=3,  
    cluster_std=1.2,  
    random_state=42  
)
```

```
# Step 2: Apply K-Means clustering
```

```
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)  
clusters = kmeans.fit_predict(X)
```

```
# Step 3: Visualize clusters [file:139]
```

```
plt.figure(figsize=(10, 6))  
plt.scatter(X[:, 0], X[:, 1], c=clusters, cmap='viridis', s=50, alpha=0.7)  
plt.scatter(kmeans.cluster_centers_[0], kmeans.cluster_centers_[0],  
            color='red', marker='X', s=200, linewidths=3, label='Centroids')  
plt.title("K-Means Clustering - Synthetic Dataset (3 Clusters)")  
plt.xlabel("Feature 1")  
plt.ylabel("Feature 2")  
plt.legend()  
plt.grid(True, alpha=0.3)  
plt.tight_layout()  
plt.show()
```

```
# Step 4: Export clustered data
```

```
df = pd.DataFrame(X, columns=['Feature1', 'Feature2'])  
df['Cluster'] = clusters  
df.to_csv("clustered_data.csv", index=False)
```

```
print("Clustered data exported to 'clustered_data.csv'")

print("Dataset shape:", df.shape)

print("\nCluster distribution:\n", df['Cluster'].value_counts().sort_index())
```

Procedure

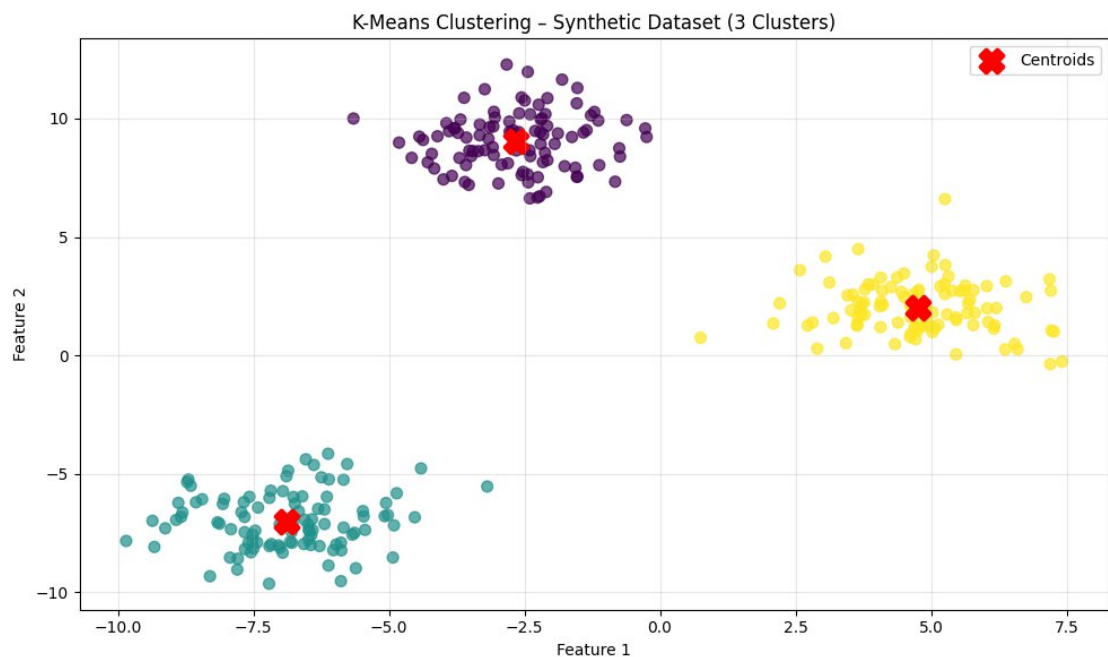
Python – Generate and cluster dataset

1. Generated synthetic 2D dataset of 300 points in 3 clusters using `sklearn.datasets.make_blobs` with `cluster_std=1.2`.
2. Applied K-Means clustering (`KMeans(n_clusters=3)`) to assign each point to the nearest centroid.
3. Created scatter plot visualization with points colored by cluster labels (purple/yellow/cyan) and red X centroids marking cluster centers.

Output

Python outputs:

- K-Means scatter plot showing 3 distinct clusters (purple: ~102 points left, cyan: ~85 points bottom-left, yellow: ~113 points right) with 3 red centroid X markers.



- Terminal output:

Clustered data exported to 'clustered_data.csv'

Dataset shape: (300, 3)

Cluster distribution:

0 102

1 85

2 113

Name: Cluster, dtype: int64

Observations

1. The scatter plot successfully identifies 3 well-separated clusters in 2D space with minimal overlap, visualized in purple (Cluster 0, left), cyan (Cluster 1, bottom-left) and yellow (Cluster 2, right).
2. Red X centroids accurately represent the center of each cluster, confirming K-Means convergence.
3. Cluster sizes are balanced (Cluster 0: 102 points, Cluster 1: 85, Cluster 2: 113), showing even partitioning of the 300 data points.

Conclusion

K-Means clustering was successfully implemented on a synthetic 2D dataset (300 points, 3 clusters), with clear visualization showing well-separated clusters and accurate centroids., enabling detailed analysis of feature distributions within each cluster and demonstrating the complete workflow from generation to visualization.